#3

Programs, Instructions, and Registers

CS 0447
Introduction to
Computer Programming

Luís Oliveira

Original slides by: Jarrett Billingsley
Modified with bits from: Bruce Childers, David Wilkinson

Fall 2020

What do I need to know now!

<u>The classes will be recorded!</u>

- You will be able to access the videos online
  - They are for your personal use only!
  - Do not distribute them!

- You don't need to turn on your camera
  - If you do, you may be recorded

- You can ask questions via text!
  - Chat is great for that. If I don't stop and read your questions, ask them again
  - But feel free to interrupt me at any point.

Class announcements

- Don’t forget!
  - Use the MARS I have on the course website:

- It has been modified!!

Programs and Instructions

What are they?

- An **instruction** is a single, simple operation for the computer to carry out, such as:
  - "add two numbers together"
  - “copy a number from one location to another"
  - "go to a different place in the program"
  - "search a string for a character“

- A **program** is a series of these tiny instructions
  - How do we create these instructions?
  - And how does the computer "understand" them?
    - does the computer understand anything?

Machine language and Assembly language

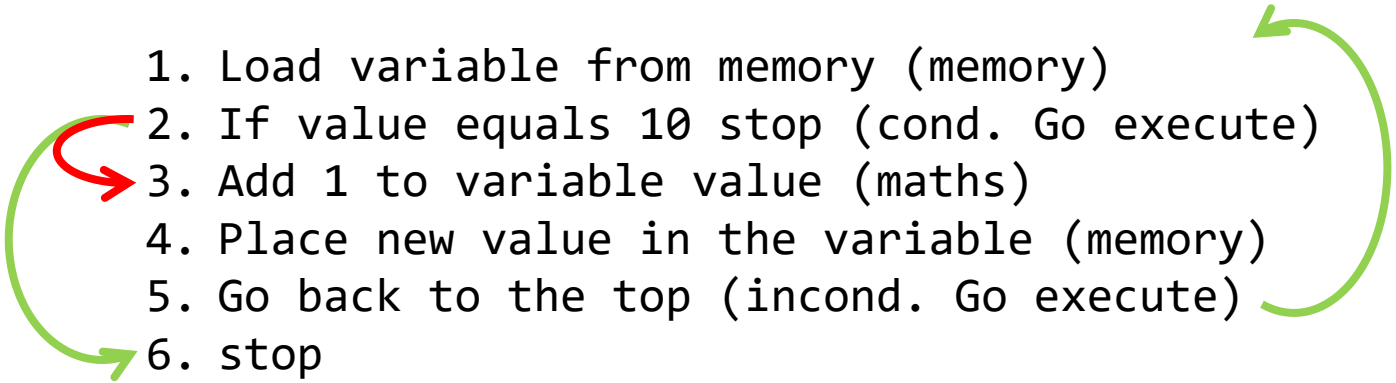
- **Machine language instructions** are the patterns of bits that a processor reads to know what to do
- **Assembly language** (or "**asm**") is a human-readable, textual representation of machine language

MIPS asm	MIPS machine language
`lw t0, 1200(t1)`	`100011 01001 01000 0000010010110000` `lw t1 t0 1200`
`add t2, s2, t0`	`000000 10010 01000 01010 00000 100000` `<math> s2 t0 t2 n/a add`
`sw t2, 1200(t1)`	`101011 01001 01010 0000010010110000` `sw t1 t2 1200`

Maths	Load/Store things from/to memory	Go execute somewhere else

Example: Count up to 10

1. Load variable from memory (memory)
2. If value equals 10 stop (cond. Go execute)
3. Add 1 to variable value (maths)
4. Place new value in the variable (memory)
5. Go back to the top (incond. Go execute)
6. stop



IS THAT ENOUGH?

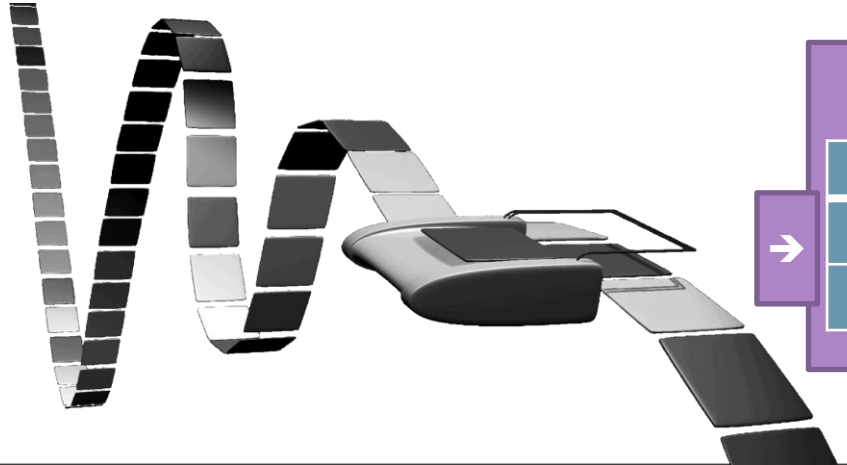
Remember the Turing machine?

CPUs are WAY more complex

- Has infinite memory represented by a single tape.
  - A head moves along the tape and can read and write values.
    - The movement (left or right) is based upon the value read and the state of the machine.

- The machine:
  1. Reads/writes the memory (tape)
  2. Compares that data and decides where to move (execute) next

- Everything that can be computed, is computed by a Turing machine



Rulebook 1

	Read	Write	Move	Next
→	0	1	←	20
	1	0	→	12

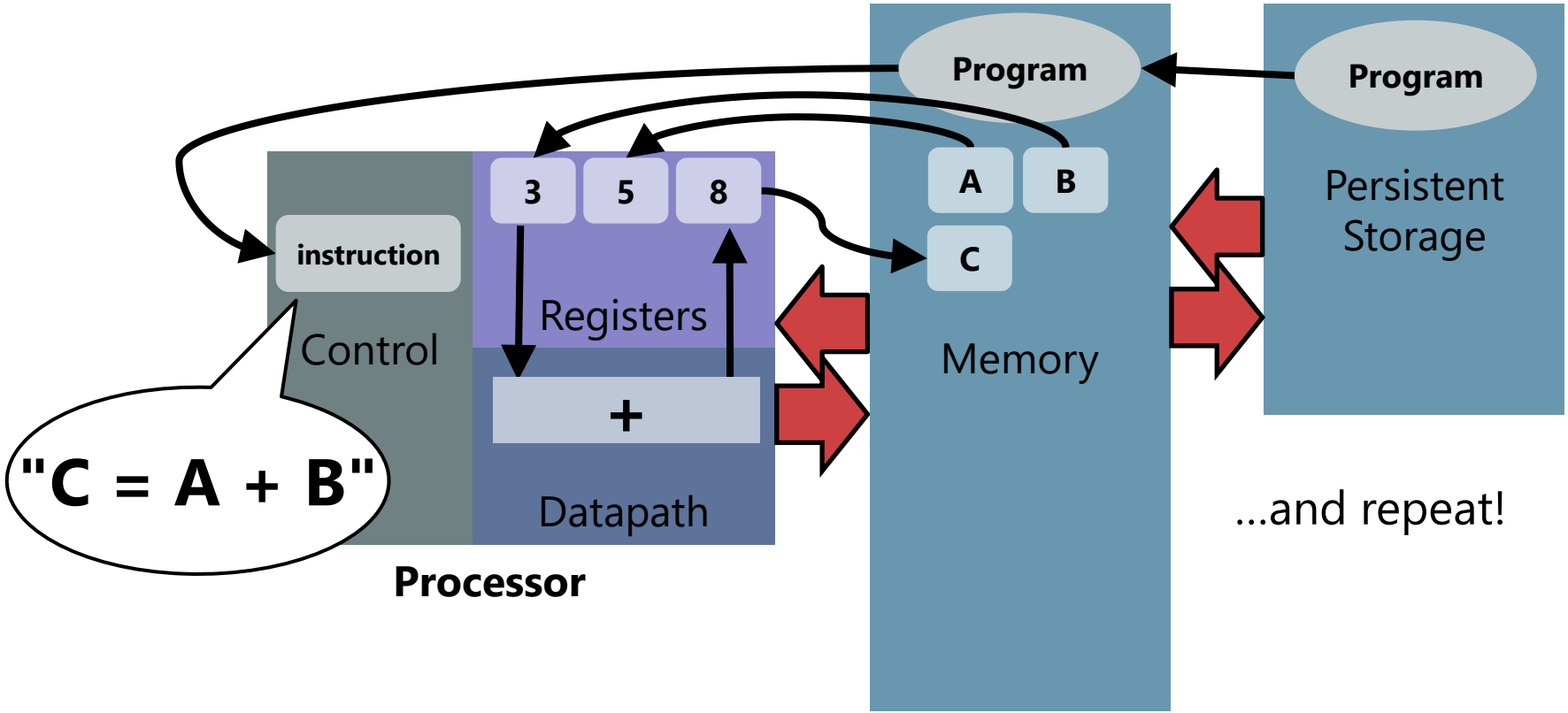
Rulebook 20

Read	Write	Move	Next
0	0	→	15
1	1	→	15

How a CPU runs a program

1. read an instruction
2. do what it says
3. go to step 1
   - ...okay there's a little more to it than that

How a CPU runs a program



ISAs

Instruction Set Architecture (ISA)

- An **ISA** is **the interface that a CPU presents to the programmer**
  - When we say "architecture", *this* is what we mean

- ISAs define:
  - **WHAT** the CPU **can do** (add, subtract, call functions, etc.)
  - **WHAT** **registers** it has (we'll get to those)
  - **WHAT** the **machine language** is
    - Machine language: the bit patterns used to encode instructions

- ISAs **do not** define:
  - **HOW** the CPU does it
  - **HOW** to design the hardware!
    - ...if there's any hardware at all
      – Java

WHY? O.o

ISAs example: x86

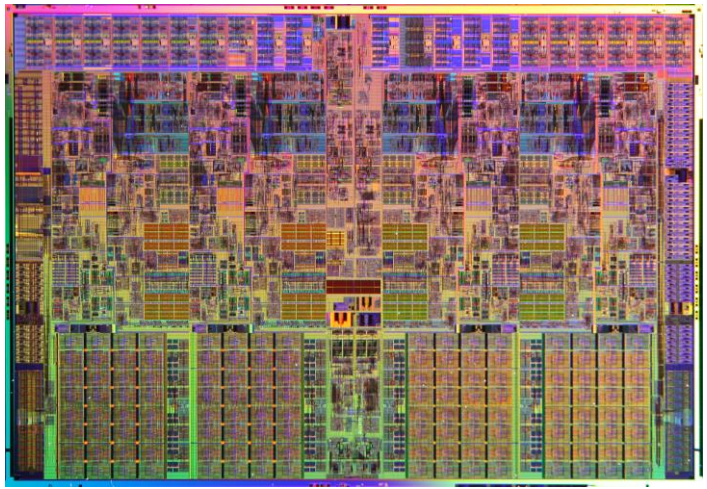
- Descended from 16-bit 8086 CPU from ***1978***
  - *Implemented in a rush by intel*
- Extended to 32 bits, then 64
- Each version can **run all programs** from the previous version
  - you can run programs written in 1978 on a brand new CPU!

- So why don't we learn x86 in this course?
  - It can do *a lot* of things
  - Its machine language is very complex
  - Making an x86 CPU is… difficult
  - Ultimately, **we would waste a ton of time**



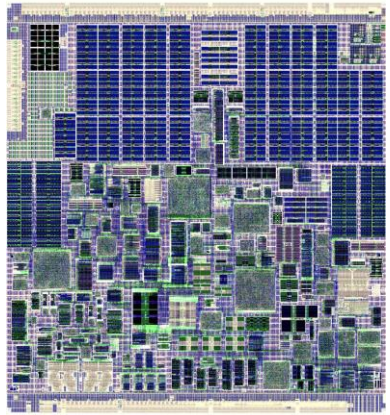
All three processors run the exact same programs...

- but they're TOTALLY different on the inside

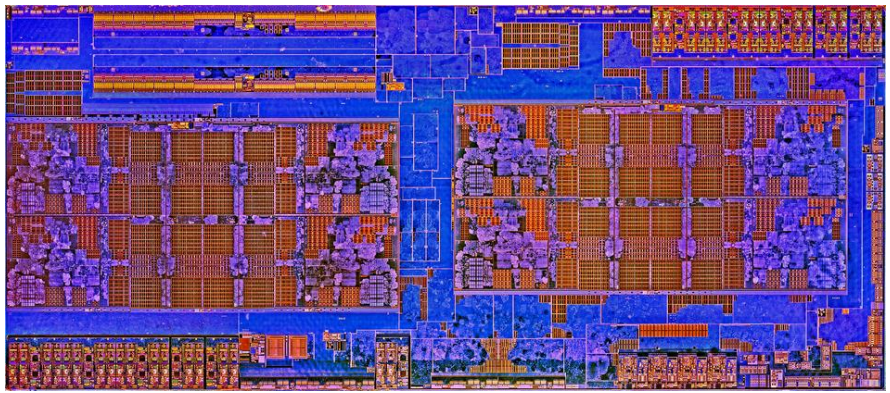


Intel Core i7

VIA Nano



AMD Zen



Kinds of ISAs: CISC

- CISC: "Complex Instruction Set Computer"
- ISA designed for humans to write asm
  - from the days before compilers!
- lots of instructions and ways to use them
- complex (multi-step) instructions to shorten and simplify programs
  - "search a string for a character"
  - "copy memory blocks"
  - "check the bounds of an array access"
- x86 is very CISCy



Kinds of ISAs: RISC

- RISC: "Reduced Instruction Set Computer"
- ISA designed to make it easy to:
  - build the CPU hardware
  - make that hardware run fast
  - write compilers that make machine code
- a small number of instructions
- instructions are very simple
- MIPS is very RISCy
- MIPS and RISC were the original RISC architectures developed at two universities in California
  - the research leads were… Patterson and Hennessy…



Popular ISAs today

- **x86** (these days, it's **x86-64** or "**amd64**")
  - most laptops/desktops/servers have one
  - (modern x86 CPUs are just RISC CPUs that can read the weird x86 instructions)
    - (unless you ask Intel, they will say otherwise ☺)
- **ARM**
  - almost everything else has one
  - ARMv8 (**AArch64**) is pretty similar to MIPS!
    - More than to ARMv7: "the main similarity between ARMv7 and ARMv8 is the name" – Comp. Org. & Design page 159
- Everything else: Alpha, Sparc, POWER/PPC, z, z80, 29K, 68K, 8051, PIC, AVR, Xtensa, SH2/3/4, 68C05, 6502, SHARC, MIPS...
  - microcontrollers, mainframes, some video game consoles, and historical/legacy applications
- despite its limited use today, MIPS has been incredibly influential!

The MIPS ISA: Registers

The registers

- **Registers** are a small and **fast** temporary memory inside the CPU
- The CPU can **only operate** (add, etc.) **on data in registers**
- MIPS has **32 registers**, and each is 32 bits (one **word**)
- The registers are numbered 0 to 31…
  - …but they also have nice names
    - The MARS version on the course website is modified
      - so you don't have to use them $ signs in the registers
      - $s0, $t1 vs. s0, t1

Used for multiplication (more on that later)

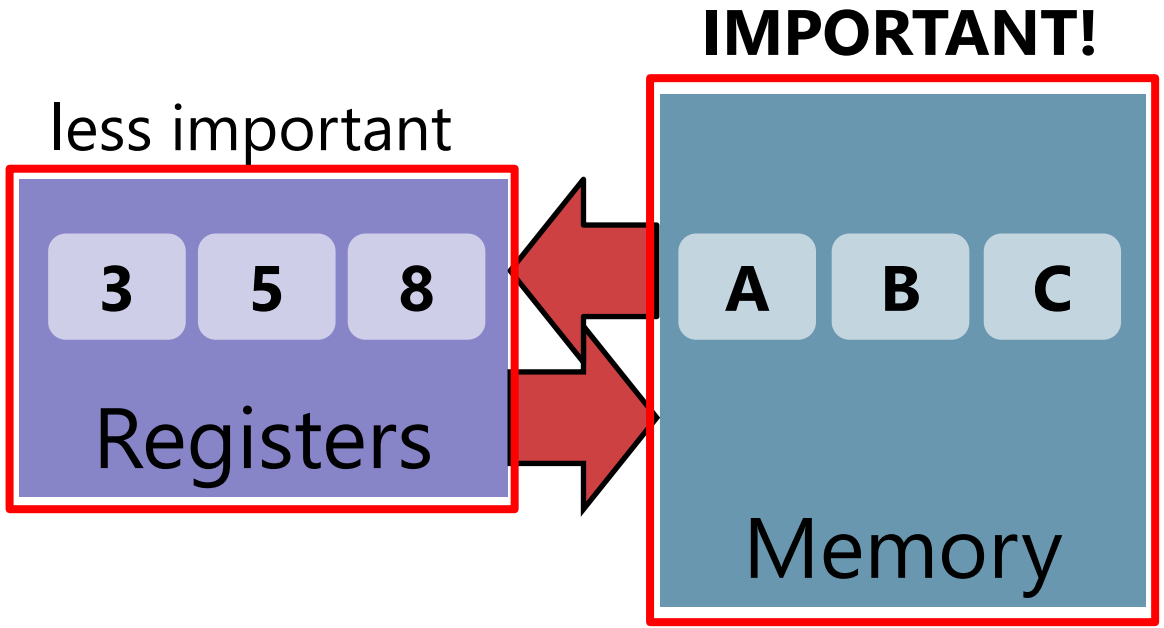
Keeps track of the next instruction to be executed

Special purpose
HI
LO
PC

General Purpose		
	#	Name
Erm…	0	zero
Avoid Totally	1	at
Used for functions	2, 3	v0, v1
	4..7	a0..a3
Used for almost everything else	8..15	t0..t7
	16..23	s0..s7
	24, 25	t8, t9
Don't need these	26, 27	k0, k1
	28	gp
For later ;)	29	sp
Don't matter	30	fp
Also for later ;)	31	ra

The juggler

- Registers are… like….. hands
- **You have a limited number and they can only hold small things**
- Your program's variables primarily live **in memory**
- The registers are just **a temporary stopping point** for those values



Really, you don't have that many

- **You cannot write every program using only registers**
  - Don't try to
    - please.
- **Every piece of your program has to SHARE the registers.**
  - Unlike high-level languages
  - Where everyone gets their own locals
  - Not in assembly!

The *s* (saved) and the t(temporary) registers

- There are ten temporary registers, **t0** through **t9**
  - These are used for *temporary* values – values that are used briefly

Name
`t0..t9`

- There are 8 saved registers, **s0** through **s7**
  - These are *kinda* like… local variables inside a function

Name
`s0..s7`

When to use each

- We'll learn more about this in the coming weeks
- Rule of thumb:
  - Use t register
  - **Unless** you *need* the value to persist **when calling functions**
    - ok that's not too clear yet

- **90%** (made up percentage) of your code will use s and t registers

The MIPS ISA: WHAT can it do?

We have a semester to learn ;)

For now:

li → **Loads a number (Immediate)**

add → **It adds 2 numbers**

sub → **It subtracts 2 numbers**

mul → **It multiplies 2 numbers**

div → **It multiplies 2 numbers**

move → **It … ermmm… COPIES a number**

Example: Loading immediates and adding them

```
s0 = 3;          li  s0, 3
s1 = 5;          li  s1, 5
s2 = s0 + s1;    add s2, s0, s1
```

- **li** stands for "load immediate." what does it look like it does?
  - "immediate" means "number inside the instruction"
- **add**, uh, makes coffee. ¬_¬
- Just like in Java, C, whatever: the **destination** is on the left

Example: Complex expression

- We can re-use registers (**t0 in the example**) as a **temporary**
  - For example, say we had a longer expression:

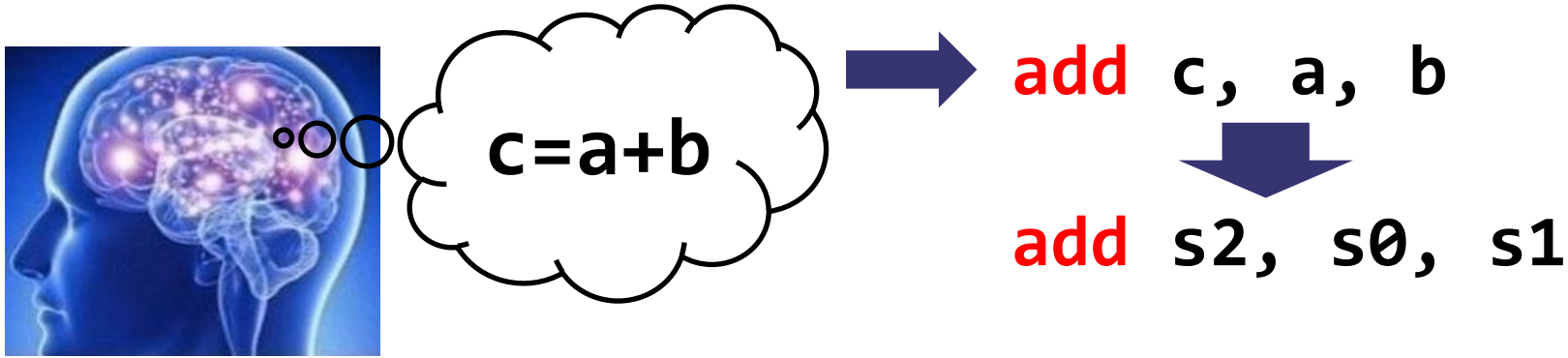
```
s4 = (s0 + s1 - s2) * s3
```

- What does algebra say about what order we should do this in?

```
add t0, s0, s1
sub t0, t0, s2
mul s4, t0, s3
```

You will be thinking like a compiler

- Writing **ASM** is a different way of programming than you're used to
- To make the transition easier, try to **reduce your cognitive load**
  - **cognitive load** is "the set of ideas you have to keep in your mind to perform some task."
  - **high-level languages (HLLs)** reduce cognitive load by hiding the machine code, using a compiler to write it for you
- you can do the same thing: think about how to write a program in e.g. C, and then turn *that* into asm



The other way around

- going the other way is also useful

how would we write this in C/Java?

```
mul t0, s2, 33   t0 = s2 * 33
div t1, s3, s4   t1 = s3 / s4
sub s1, t0, t1   s1 = t0 - t1
```

or, if we rolled it all together,

```
s1 = (s2 * 33) - (s3 / s4)
```

that's what this asm *does*

Why do you need to know this?

- CS0447 is about **building a mental model of how a computer works**

- Understanding what is happening when you write code or run programs gives you a much deeper understanding
  - "why should I avoid using this programming language feature in this speed-critical part of my code?"
  - "why wouldn't this crazy idea be very fast on current architectures?"
  - "this program is breaking in a really confusing way, I have to look at the asm to debug it"

- This stuff is specialized but hey you're majoring/minoring in it right